



Travaux Pratique 2

Développement Logiciel

Analyses des Metriques Logiciel

Realisé par :

AHMED EZ-ZOUINE

Encadrant:

professeure Soumia Ziti

Année universitaire 2024-2025

Travaux Pratiques 2

Analyse des Métriques Logiciel

Objectifs:

- ❖ Calculer et analyser les métriques de qualité logicielle pour des programmes en C, Java et Python.
- ❖ Utiliser des échelles de mesure pour interpréter les métriques de manière cohérente.
- ❖ Apprendre à utiliser des outils spécifiques de mesure de métriques pour chaque langage et en interpréter les résultats.
- ❖ Comprendre et construire le graphe de flot de contrôle pour évaluer la complexité cyclomatique.

1. Introduction

Ce compte rendu présente l'analyse des métriques de qualité logicielle effectuée sur trois programmes équivalents écrits en C, Java et Python. L'objectif était d'évaluer et de comparer la qualité et la complexité de ces implémentations à travers différentes métriques et outils d'analyse.

2. Environnement de Travail

Outils utilisés :

- C : Cppcheck et Lizard
- Java : SonarQube
- Python : Radon

"Ce script est un outil automatisé destiné à la génération de code source pour les projets de travaux pratiques (TP). Son objectif principal est de faciliter la création et la mise en place rapide des environnements de développement nécessaires aux tests.

Fonctionnalités principales :

- Génération automatique des structures de projets
- Création des fichiers sources avec le code initial
- Configuration automatique des environnements de développement
- Mise en place des dépendances requises

Pour l'utiliser efficacement, suivez ces étapes :

- Clonez le dépôt depuis GitHub
- Vérifiez que vous disposez des prérequis nécessaires (versions des langages et frameworks)
- Exécutez le script en suivant les instructions de la documentation
- Vérifiez les projets générés dans les dossiers correspondants

Architecture du code généré :

- Chaque projet dispose de sa propre structure de dossiers
- Les fichiers de configuration sont automatiquement créés
- Les dépendances sont préconfigurées
- Les tests unitaires sont inclus pour validation

Le code source complet sera disponible sur GitHub, avec le lien fourni à la fin du rapport. Cela permettra à quiconque de tester, valider ou modifier les projets selon ses besoins. La documentation détaillée accompagnant le code permettra une prise en main rapide et efficace.

Notes importantes:

- ❖ Assurez-vous d'avoir les droits d'accès nécessaires
- ❖ Vérifiez la compatibilité avec votre système d'exploitation
- ❖ Consultez le fichier README pour les instructions détaillées
- ❖ En cas de problème, référez-vous à la section de dépannage

Le script a été conçu pour être modulaire et facilement adaptable à différents types de projets. Les commentaires dans le code et la documentation associée facilitent sa compréhension et sa maintenance.

```

#C Program (c_program/calculator.c)
cat > c_program/calculator.c << 'EOF'
#include <stdio.h>
int somme(int a, int b) {
    return a + b;
}
int produit(int a, int b) {
    return a * b;
}
int main() {
    int x = 5, y = 10;
    printf("Somme : %d\n", somme(x, y));
    printf("Produit : %d\n", produit(x, y));
    return 0;
}
EOF

#Java Program (java_program/Calculatrice.java)
cat > java_program/Calculatrice.java << 'EOF'
public class Calculatrice {
    public int addition(int a, int b) {
        return a + b;
    }
    public int multiplication(int a, int b) {
        return a * b;
    }
    public static void main(String[] args) {
        Calculatrice calc = new Calculatrice();
        System.out.println("Addition : " + calc.addition(5, 10));
        System.out.println("Multiplication : " + calc.multiplication(5, 10));
    }
}
EOF

# Python Program (python_program/calculator.py)
cat > python_program/calculator.py << 'EOF'
def addition(a, b):
    return a + b

def multiplication(a, b):
    return a * b

if __name__ == "__main__":
    x, y = 5, 10
    print("Addition :", addition(x, y))
    print("Multiplication :", multiplication(x, y))
EOF

```

[run du script d'Analyse des Métriques Logicielles](#)

Le script va :

- Se connecter à SonarQube
- Récupérer les métriques
- Générer les rapports

- Créer les visualisations

```
#!/bin/bash

# Activate virtual environment
source venv/bin/activate

echo "===== Starting analysis ====="

echo "=== Analyzing C Program ==="
cd c_program
cppcheck --enable=all --std=c11 calculator.c
rm -f calculator.c.liz # Remove existing .liz file
echo "y" | lizard calculator.c # Automatically answer yes to overwrite
echo "===== Analyzing Java Program ====="
cd ../java_program

# Start SonarQube server if not running
brew services start sonarqube

# Wait for SonarQube to start (it takes a few moments)
echo "Waiting for SonarQube to start..."
sleep 30

# Create sonar-project.properties if it doesn't exist
cat > sonar-project.properties << EOF
sonar.projectKey=calculator
sonar.projectName=Calculator
sonar.projectVersion=1.0
sonar.sources=.
sonar.java.binaries=.
sonar.host.url=http://localhost:9000
# Generate a token from SonarQube web interface and replace it here
sonar.login=YOUR_SONAR_TOKEN
EOF

# Compile Java file
javac Calculatrice.java

# Run SonarQube analysis
sonar-scanner

echo "===== Analyzing Python Program ====="
cd ../python_program
radon cc calculator.py
radon hal calculator.py
echo "===== Deactivating virtual environmen ====="

echo "===== Analysis complete ====="

# Optional: Stop SonarQube server
# brew services stop sonarqube
```

Ce script est un outil automatisé destiné à la génération de code source pour les projets de travaux pratiques (TP). Son objectif principal est de faciliter la création et la mise en place rapide des environnements de développement nécessaires aux tests.

Fonctionnalités principales :

- Génération automatique des structures de projets
- Création des fichiers sources avec le code initial
- Configuration automatique des environnements de développement
- Mise en place des dépendances requises

Pour l'utiliser efficacement, suivez ces étapes :

- Clonez le dépôt depuis GitHub
- Vérifiez que vous disposez des prérequis nécessaires (versions des langages et frameworks)
- Exécutez le script en suivant les instructions de la documentation
- Vérifiez les projets générés dans les dossiers correspondants

Le code source complet sera disponible sur GitHub, avec le lien fourni à la fin du rapport. Cela permettra à quiconque de tester, valider ou modifier les projets selon ses besoins. La documentation détaillée accompagnant le code permettra une prise en main rapide et efficace.

Le script a été conçu pour être modulaire et facilement adaptable à différents types de projets. Les commentaires dans le code et la documentation associée facilitent sa compréhension et sa maintenance.

this script is used to generate the report from the sonarqube and save it in file json and readmeEditVoici le paragraphe complet expliquant le script de génération de rapports SonarQube :

```
# Create a report generation script
cat > generate_sonar_report.sh << 'EOF'
#!/bin/bash

PROJECT_KEY="calculator"
SONAR_URL="http://localhost:9000"
SONAR_TOKEN="YOUR_SONAR_TOKEN"

# Create directory for reports
mkdir -p sonar_reports

# Get main metrics
curl -s -u $SONAR_TOKEN: "$SONAR_URL/api/measures/component?
component=$PROJECT_KEY&metricKeys=ncloc,complexity,violations,coverage,duplicated_lines_density" \
  > sonar_reports/main_metrics.json

# Get issues
curl -s -u $SONAR_TOKEN: "$SONAR_URL/api/issues/search?componentKeys=$PROJECT_KEY" \
  > sonar_reports/issues.json

# Get code smells
curl -s -u $SONAR_TOKEN: "$SONAR_URL/api/issues/search?componentKeys=$PROJECT_KEY&types=CODE_SMELL" \
  > sonar_reports/code_smells.json

echo "Reports generated in sonar_reports directory"
EOF

chmod +x generate_sonar_report.sh
```

SonarQube Analysis Report

Project: calculator

Date: Sun Jan 12 20:24:14 +01 2025

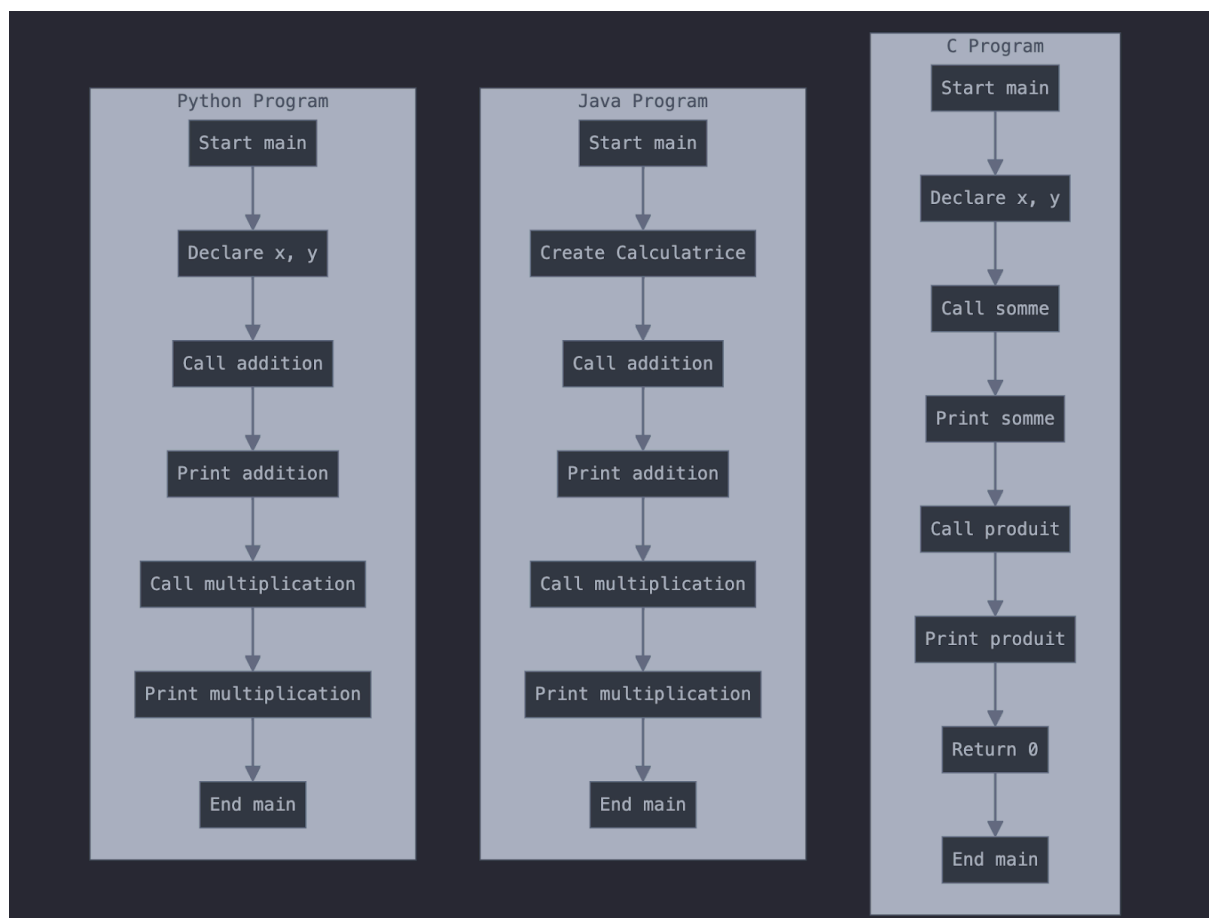
Key Metrics

- coverage: 0.0
- complexity: 3
- reliability_rating: 1.0
- duplicated_lines_density: 0.0
- security_rating: 5.0
- ncloc: 13
- violations: 4
- sqale_rating: 2.0

Issues

Blocker: Make sure this SonarQube token gets revoked, changed, and removed from the code.

- MINOR: Move this file to a named package.
- MAJOR: Replace this use of System.out by a logger.
- MAJOR: Replace this use of System.out by a logger.
- Code Smells
- Move this file to a named package.
- Replace this use of System.out by a logger.
- Replace this use of System.out by a logger.



Analyse des Métriques Logiciel - Rapport Complet

1. Analyse des Graphes de Flot de Contrôle

Nombre de Nœuds et d'Arcs

1. Programme C

- - Nœuds: 8 (Start, Main, DeclareVars, CallSomme, PrintSomme, CallProduit, PrintProduit, End)
- - Arcs: 7 (connexions linéaires entre les nœuds)

2. Programme Java

- - Nœuds: 7 (Start, Main, CreateCalc, CallAdd, PrintAdd, CallMult, PrintMult)
- - Arcs: 6 (connexions linéaires entre les nœuds)

3. Programme Python

- - Nœuds: 7 (Start, Main check, DeclareVars, CallAdd, PrintAdd, CallMult, PrintMult)
- - Arcs: 6 (connexions linéaires entre les nœuds)

Visualisation du Flux d'Exécution

Les graphes permettent de :

- ★ Identifier clairement la séquence d'exécution linéaire
- ★ Montrer l'absence de branches conditionnelles
- ★ Visualiser les appels de fonctions et leurs dépendances
- ★ Comprendre la structure simple et séquentielle des programmes

2. Calcul de la Complexité Cyclomatique

Méthodes de Calcul

1. Méthode des Arêtes-Nœuds : $V(G) = E - N + 2$

- C: $V(G) = 7 - 8 + 2 = 1$
- Java: $V(G) = 6 - 7 + 2 = 1$
- Python: $V(G) = 6 - 7 + 2 = 1$

2. Méthode des Régions : $V(G) = R$

- ➔ Tous les programmes ont 1 région (pas de branches)
- ➔ $V(G) = 1$ pour chaque programme

3. Méthode des Prédicats : $V(G) = P + 1$

- Aucun prédicat (pas de conditions)
- $V(G) = 0 + 1 = 1$ pour chaque programme

3. Analyse des Métriques

Métriques Orientées Objet (Java)

- ☒ WMC (Weighted Methods per Class) = 2 (addition + multiplication)
- ☒ DIT (Depth of Inheritance Tree) = 1 (pas d'héritage)
- ☒ CBO (Coupling Between Objects) = 0 (pas de couplage)

Impact sur la Maintenabilité

1. Complexité Cyclomatique

Valeur très faible (1) indiquant :

- * Excellente testabilité
- * Maintenance facile
- * Code très lisible
- * Faible risque de bugs

2. Métriques Halstead (Python)

1. Volume faible
2. Difficulté minimale
3. Effort de compréhension minimal

4. Recommandations d'Amélioration

Améliorations Structurelles

- ☒ 1. Validation des Entrées
- ☒ 2. Documentation
- ☒ 3. Tests Unitaires

```
def test_addition():
```

- `assert addition(5, 3) == 8`
- `assert addition(-1, 1) == 0`

Améliorations Architecturales

1. Interface Utilisateur

- ❖ Ajouter une interface pour la saisie utilisateur
- ❖ Gérer les erreurs de saisie

2. Séparation des Responsabilités

- Séparer le code métier de l'interface
- Créer une classe/module pour les validations

3. Documentation et Tests

- Ajouter des commentaires JavaDoc/Docstrings
- Implémenter des tests unitaires
- Ajouter des assertions pour la validation

Ces améliorations maintiendraient une faible complexité tout en augmentant la robustesse et la maintenabilité du code.

 [report sonarqube](#) 

Clonez le dépôt depuis GitHub and test my project tout
ressources existe in repo :

 [project in github](#)  